Energy-Aware Adaptive IoT Sensing Using Edge Intelligence

Project Report

Course: Internet of Things and Industrial IoT

Problem Statement: 4

Team Member	Registration Number
Nikunj Srivastav	24BCS087
Nishit Gupta	24BEC034
Nitul Das	24BDS051

Table 1: Project team

This report presents our plan for Problem Statement 4. The project has not yet reached the building stage, so the contents below describe what we intend to do and how we will evaluate it.

1. Problem Statement

Most IoT devices read their sensors and transmit data at the same rate at all times, even when the monitored conditions are not changing. This wastes battery energy and network data.

The question this project investigates is: **how much energy and data can be saved without missing real problems?**

2. Proposed Approach

The device will monitor its surroundings and adjust its own activity. The ESP32 makes this decision on the device itself rather than in the cloud, which is what edge intelligence means in this project. Table 2 summarises the two operating modes.

	Normal mode	Alert mode
Sensor readings	Slow (about every 30 to 60 seconds)	Fast (about every 2 seconds, faster for shaking)
Data transmission	Rarely, as a short summary	Immediately
Processing	Minimal, with sleep between readings	Full, every reading is checked
Indicator	Green LED	Red LED

Table 2: Operating modes

The device will switch to alert mode if the temperature or humidity changes sharply, if the board is shaken, or if the test button is pressed. When the readings remain steady for a period of time, it returns to normal mode. The time values in Table 2 are starting values and will be tuned during testing.

3. Components Required

Table 3 lists the hardware and software components that will be used.

Component	Purpose
ESP32	Main controller of the device, with built-in Wi-Fi
DHT22 and 10 k-ohm resistor	Measures temperature and humidity (the resistor is needed only if the sensor board has no built-in pull-up)
MPU6050	Detects shaking and movement
INA219	Measures the actual power consumed by the device
18650 battery, holder and TP4056	Battery, holder and safe charging module; the battery is charged first and then used to run the device
MT3608 boost converter	Raises the battery voltage to a steady 5 V for the ESP32 board
Red and green LEDs with two 220 ohm resistors	Indicate the current mode
Push button	Triggers a test event
Breadboard, jumper wires, USB cable	Wiring and programming
Multimeter	Verifies the INA219 readings and the power connection
Laptop with Mosquitto (MQTT)	Receives the data over Wi-Fi and stores it with timestamps

Table 3: Components

4. Implementation Plan

The work will be carried out in the following steps.

1. Connect all the components on the breadboard and test each sensor individually.
2. Verify the power connection (battery, TP4056 and boost converter) with the multimeter before connecting the ESP32.
3. Write a basic version that reads and transmits data at a fixed rate. This is the first comparison version.
4. Write a second comparison version that switches to fast mode when a reading crosses a fixed limit, for example a temperature value.
5. Write the proposed version that switches between normal and alert mode based on how quickly the readings change.
6. Run each version in turn on the same ESP32 for the same duration, repeating the same test events (heat, shaking, button press). Record the time of every test event so that the correct outcome is known.
7. Compare the results and document them.

5. Evaluation Method

Three versions will be compared on the same ESP32 under the same conditions, as shown in Table 4.

Version	Behaviour
Fixed speed	Reads and transmits at the same rate at all times
Fixed limits	Switches to fast mode when a reading crosses a set value
Proposed (smart)	Switches mode based on how fast the readings change, and sleeps between readings

Table 4: Versions compared

The versions will be compared using the measures in Table 5.

Measure	How it is measured
Energy used	INA219 current multiplied by voltage and time, in each mode
Energy saved	Proposed version compared with the fixed-speed version, in percent
Data packets sent	Number of messages received by the laptop
Data saved	Bytes sent compared with the fixed-speed version
Problems caught	Test events detected out of all test events created
False alarms	Alerts raised when no event occurred
Detection time	Seconds from the start of an event to the alert
Battery life	Battery capacity divided by average current (calculated, not run until empty)

Table 5: Evaluation measures

6. Expected Results

We expect the proposed version to use less energy and transmit fewer packets than the fixed-speed version while still detecting problems quickly. Slow mode may delay detection by up to one reading interval, so both the savings and the delay will be reported. The actual numbers will be reported once testing is complete.

7. Limitations and Precautions

1. **DHT22 speed limit.** The DHT22 can be read only about once every 2 seconds, so alert mode has a speed limit for temperature. The MPU6050 is faster and covers shaking.
2. **Battery voltage.** An 18650 battery provides 3.0 to 4.2 V. This is too high for the ESP32 3.3 V pin, so the battery will never be connected there. It is also too low for the 5 V pin as the battery discharges, so a boost converter will be used and the connection will be checked with the multimeter before power is applied.
3. **Sleep is required.** The ESP32 must sleep between readings, otherwise the savings will be small. The INA219 cannot measure very small deep-sleep currents, so light sleep will be used, or that part will be estimated and stated in the report.
4. **Wi-Fi energy cost.** Reconnecting to Wi-Fi consumes power, so data will be sent in small batches instead of one reading at a time.